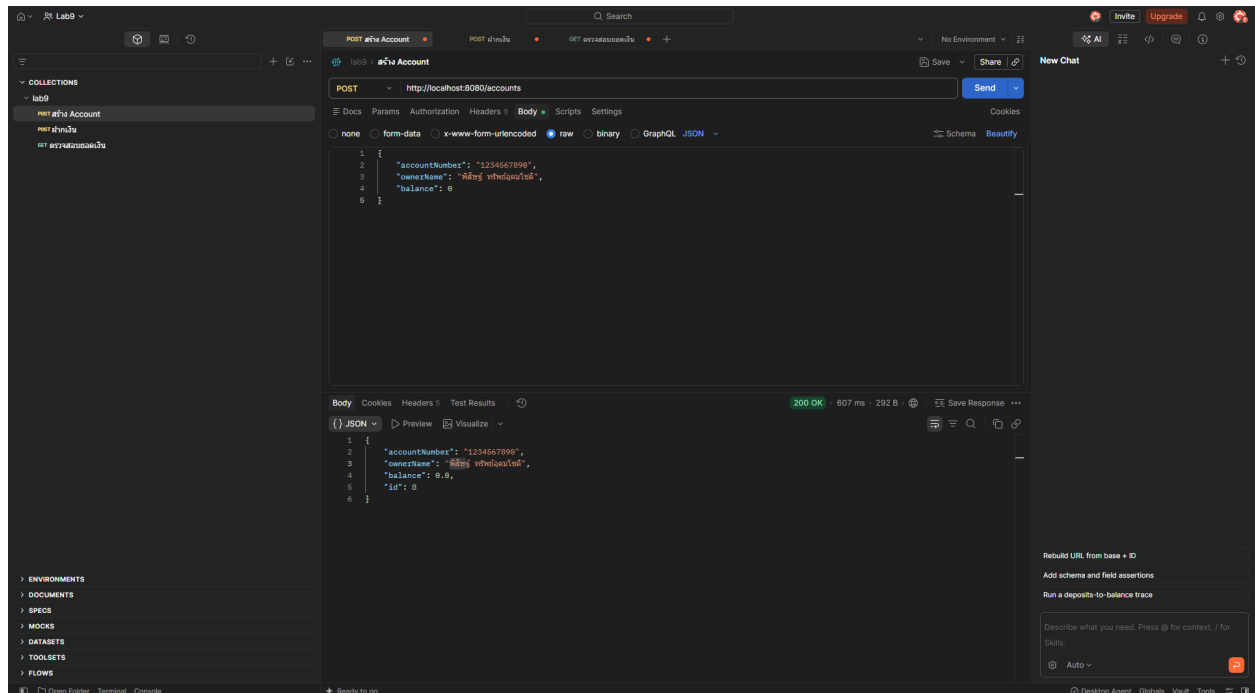


Report Lab 9: Spring Boot – Transaction

1. การสร้าง Account

Request: POST /accounts

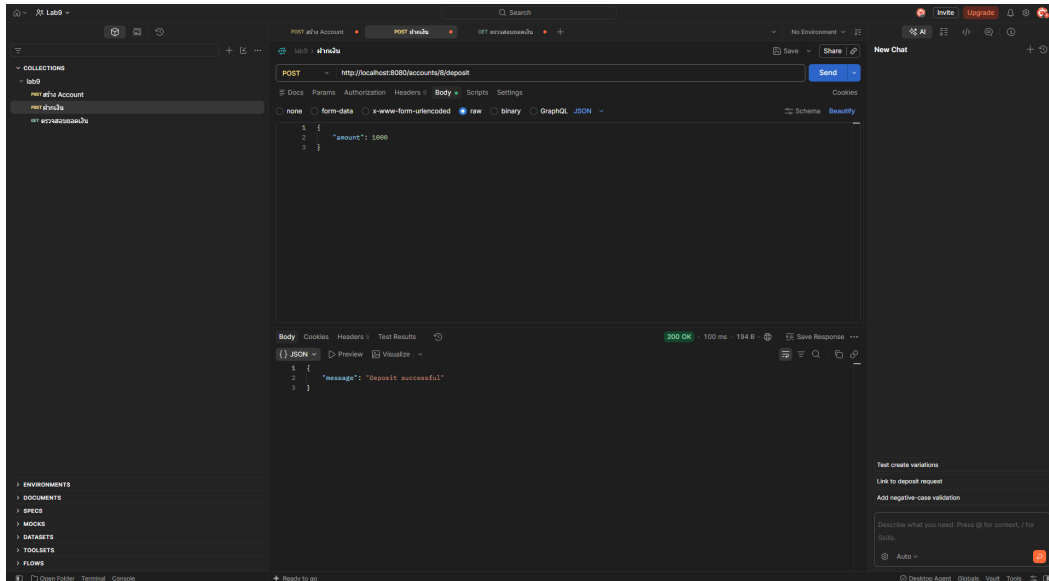


อธิบายผล: สร้างบัญชีใหม่สำเร็จ Status 200 OK และ Response ส่งค่า id: 8 กลับมาพร้อม balance เริ่มต้นเป็น 0.0 ตามที่ส่งไป แสดงว่า endpoint ทำงานถูกต้อง

พิธีษฐ ทรัพย์อุดมโชติ 673380285-2 sec.1

2. การฝากเงินสำเร็จ

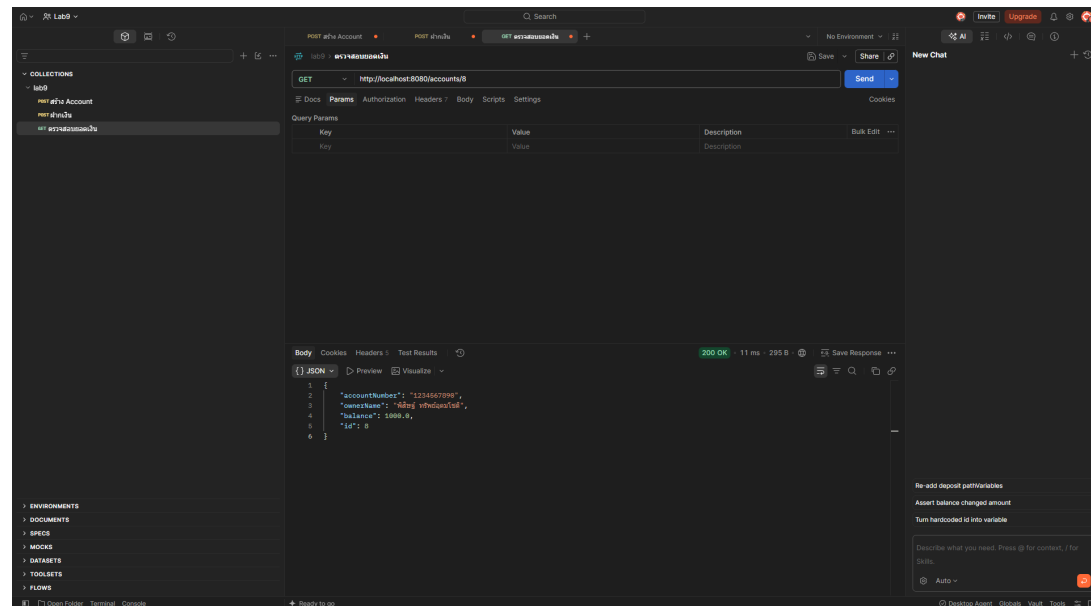
Request: POST /accounts/{id}/deposit



อธิบายผล: ส่งคำขอฝากเงินจำนวน 1000 ไปยัง Account id=8 ได้ Status 200 OK และข้อความ "Deposit successful" กลับมา แสดงว่าระบบประมวลผลการฝากเงินสำเร็จ

3. ตรวจสอบ Account

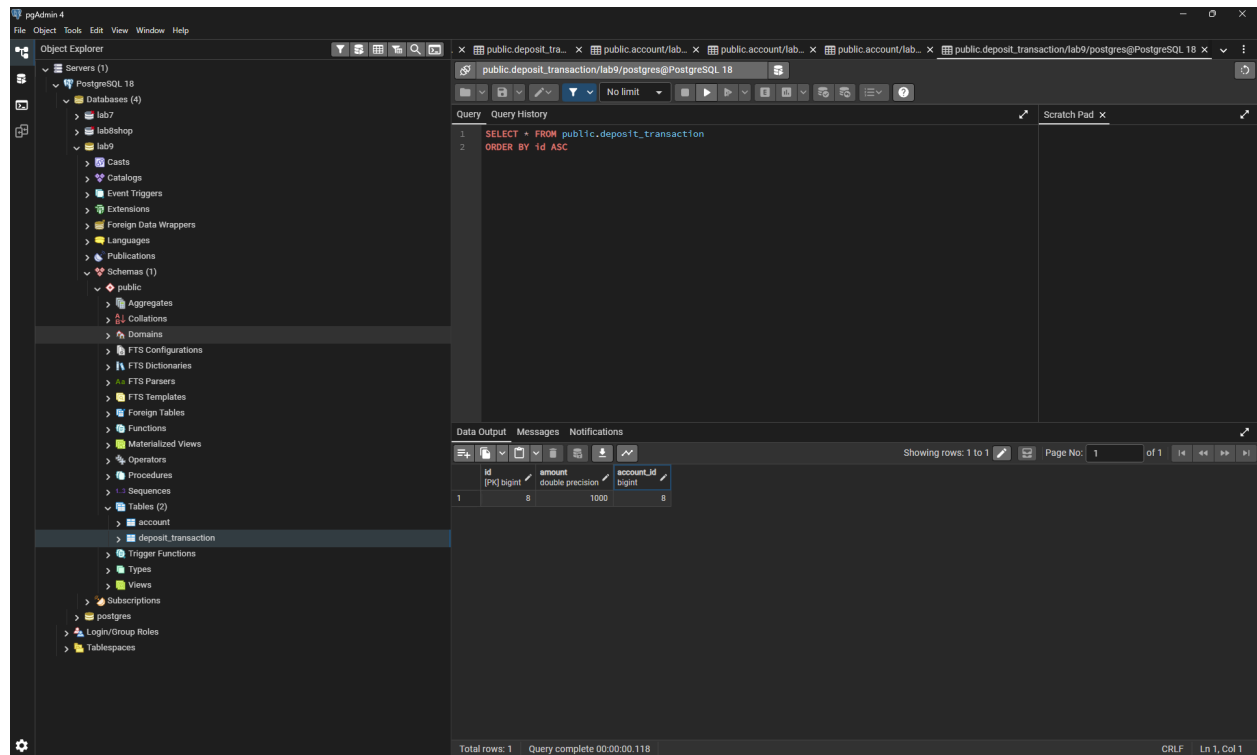
Request: GET /accounts/{id}



อธิบายผล: ดึงข้อมูล Account id=8 กลับมาดูอีกครั้ง balance จะเปลี่ยนจาก 0 เป็น 1000.0 ตรงกับจำนวนเงินที่ฝากไป ยืนยันว่ายอดเงินถูกอัปเดตในฐานข้อมูลจริง

4. ตรวจสอบ DepositTransaction

pgAdmin: SELECT * FROM deposit_transaction;

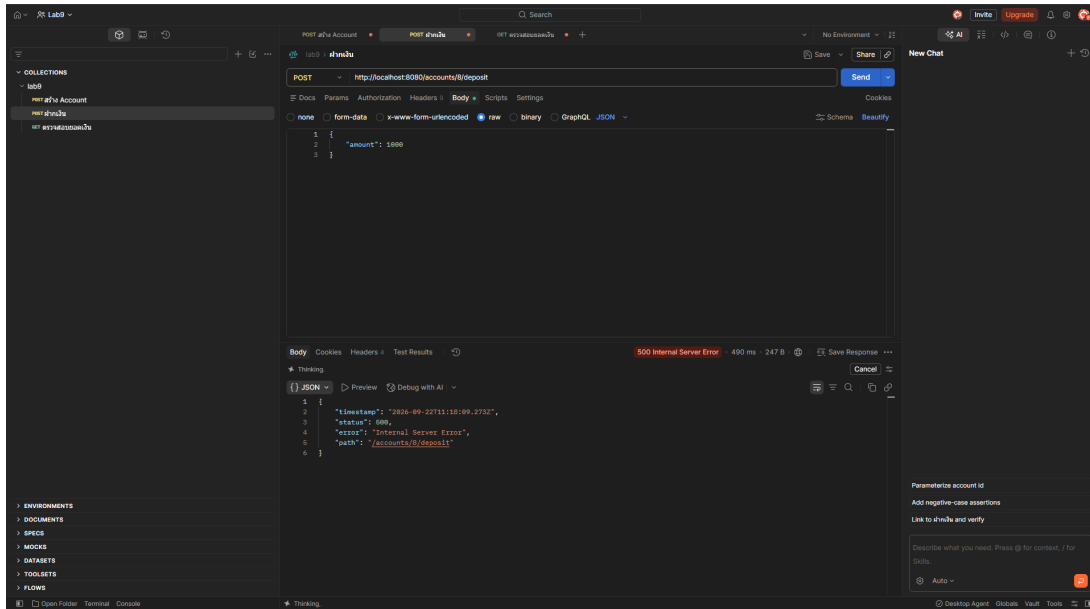


อธิบายผล: รัน SQL ตรวจสอบตาราง deposit_transaction พบ 1 แถว (amount=1000, account_id=8) แสดงว่ายอดเงินจะถูกอัปเดต ระบบยังบันทึกประวัติการฝากเงินลงตารางแยกได้ถูกต้อง

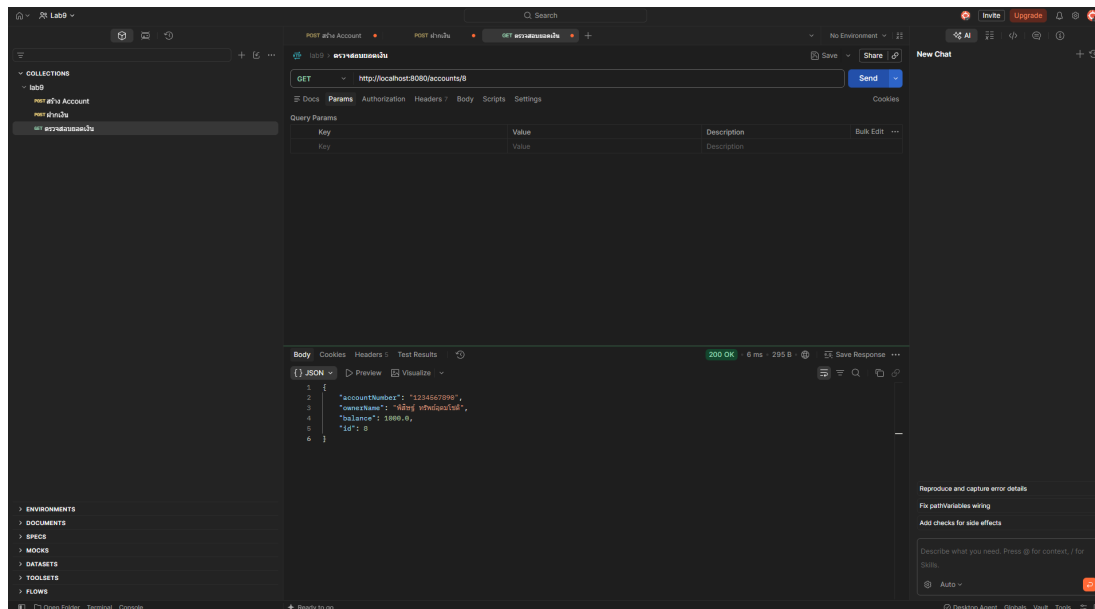
ฟิลิษฐ์ ทรัพย์อุดมโชติ 673380285-2 sec.1

5. ทดลอง Rollback

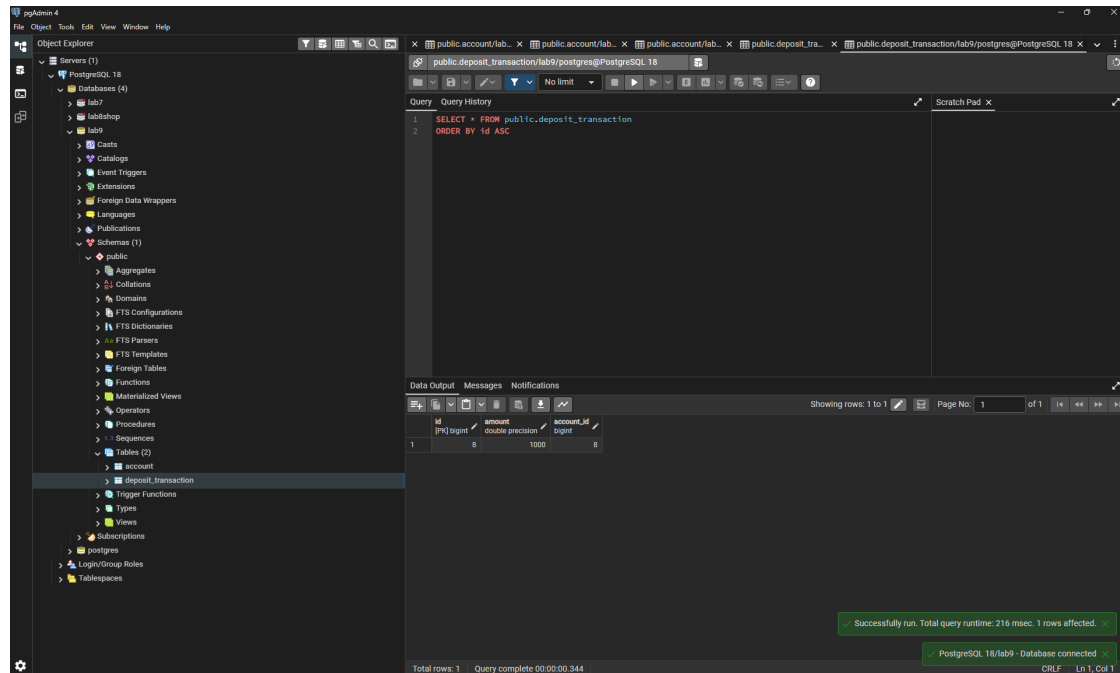
Postman แสดง Status 500 หลังเพิ่ม throw new RuntimeException



ตรวจสอบว่าข้อมูลถูกย้อนกลับ (balance ไม่เพิ่ม + deposit_transaction ไม่มีแถวใหม่)



ฟิลิษฐ์ ทรัพย์อุดมโชติ 673380285-2 sec.1

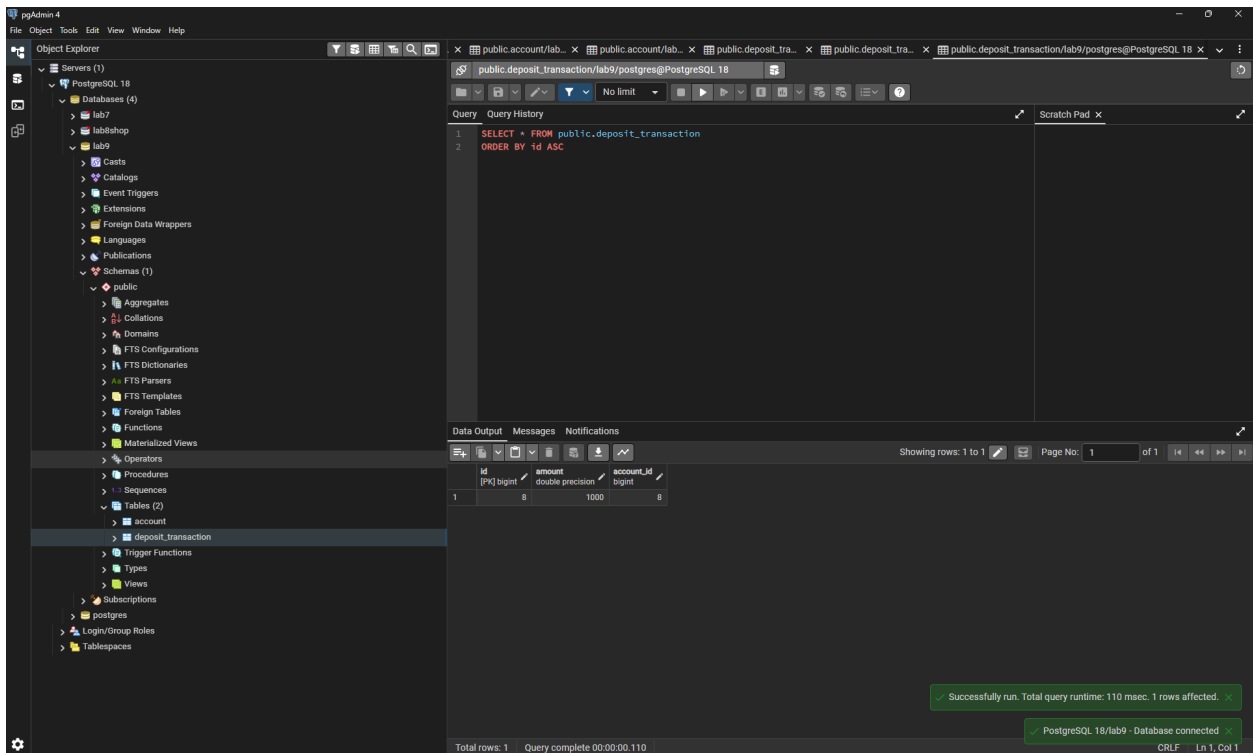
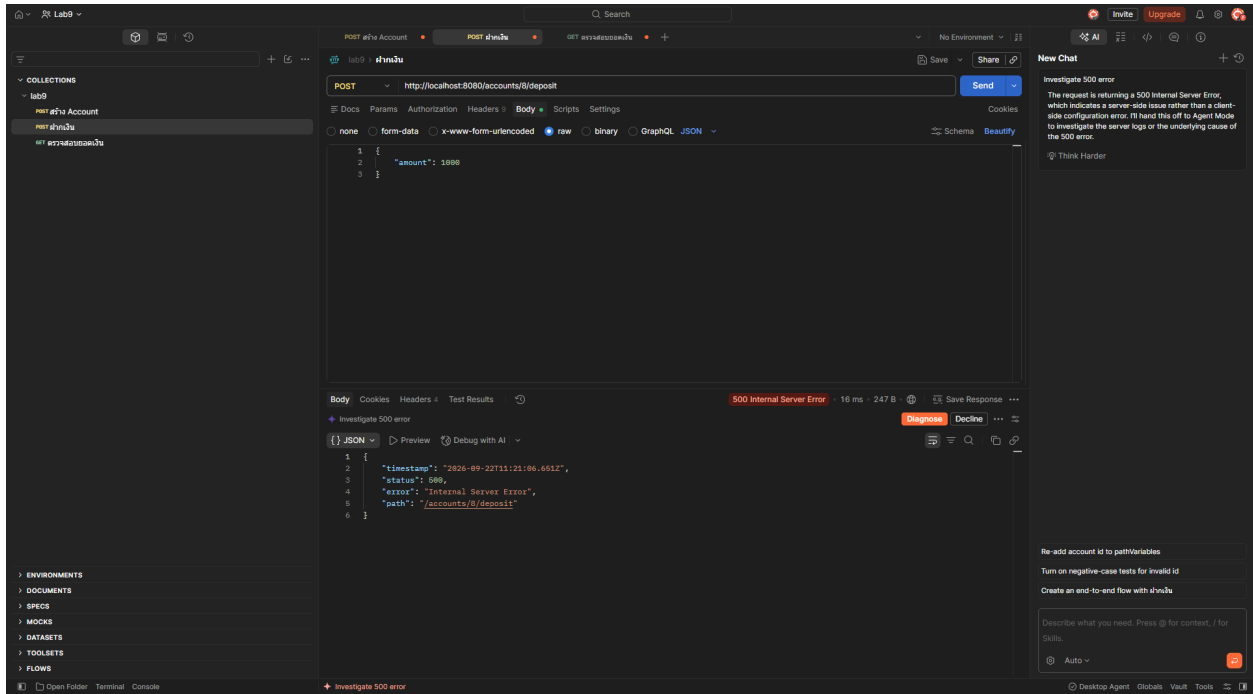


อธิบายผล: หลังเพิ่มโค้ด throw new RuntimeException("Test Rollback") ต่อท้ายเมธอด deposit() แล้วยิงคำขอฝากเงินเข้า ระบบตอบกลับ Status 500 Internal Server Error เมื่อตรวจสอบยอดเงินอีกครั้งพบว่า balance ยังคงเป็น 1000.0 เท่าเดิม และตาราง deposit_transaction ก็ยังมีแค่ 1 แถวเหมือนก่อนทดลอง แสดงว่า @Transactional ทำงานถูกต้อง

ฟิลิษฐ์ ทรัพย์อุดมโชติ 673380285-2 sec.1

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

กรณีมี @Transactional



ฟิลิษฐ์ ทรัพย์อุดมโชติ 673380285-2 sec.1

กรณีไม่มี @Transactional

COLLECTIONS

lab9

POST สร้าง Account

POST สร้างเงิน

GET ตรวจสอบยอดเงิน

POST

lab9

POST สร้างเงิน

http://localhost:8080/accounts/8/deposit

Send

Body

500 Internal Server Error

582 ms · 247 B

Save Response

Investigate 500 error

Diagnose

Decline

JSON

Preview

Debug with AI

1 {

2 "timestamp": "2026-09-22T11:23:49.766Z",

3 "status": 500,

4 "error": "Internal Server Error",

5 "path": "/accounts/8/deposit"

6 }

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

DATASETS

TOOLSETS

FLAWS

Open Folder

Terminal

Console

Investigate 500 error

Desktop Agent

Globals

Vault

Tools

pgAdmin 4

File

Object Explorer

Tools

Edit

View

Window

Help

Servers (1)

PostgreSQL 18

Databases (4)

lab7

lab8shop

lab9

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

public

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (9)

account

deposit_transaction

Trigger Functions

Types

Views

Subscriptions

postgres

Login/Group Roles

Tablespaces

public.deposit_tra...

public.deposit_tra...

public.deposit_tra...

public.deposit_tra...

public.deposit_transaction/lab9/postgres@PostgreSQL 18

Query

Query History

Scratch Pad

1 SELECT * FROM public.deposit_transaction

2 ORDER BY id ASC

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

id	amount	account_id
[PK] bigint	double precision	bigint
1	8	1000
2	9	1000

Successfully run. Total query runtime: 293 msec. 2 rows affected.

PostgreSQL 18/lab9 - Database connected

Total rows: 2 Query complete 00:00:00.293

CRLF Ln 1, Col 1

อธิบายผล:

ถ้ามี @Transactional การฝากเงินที่ล้มเหลวจะถูก ROLLBACK ข้อมูลกลับเป็นสภาพเดิม

balance และตารางประวัติจะไม่เปลี่ยนแปลง แต่หลังจากลบ @Transactional ออกแล้วทดลองฝากเงิน จะยังมี error เกิดขึ้นอยู่ (Status 500) แต่พบว่า balance กลับเพิ่มขึ้นเป็น 2000.0 และตาราง deposit_transaction มีแถวเพิ่มขึ้นเป็น 2 แถว ทั้งที่คำขอทุกครั้งจบด้วย error แสดงให้เห็นว่าเมื่อไม่มี @Transactional คำสั่ง save() แต่ละคำสั่งจะถูกบันทึกลงฐานข้อมูลทันทีแยกจากกัน ไม่มีการย้อนกลับแม้จะเกิดข้อผิดพลาดในภายหลัง ทำให้ข้อมูลในระบบไม่สอดคล้องกับความเป็นจริง ต่างจากตอนที่ @Transactional อย่างชัดเจน

7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

1. @Transactional มีหน้าที่อะไร?

กำหนดขอบเขตของ Transaction ให้ทุกคำสั่งในเมธอดนั้นทำงานเป็นก้อนเดียวกัน ถ้าทุกอย่างสำเร็จจะ COMMIT พร้อมกัน ถ้ามีจุดใดพังจะ ROLLBACK ทั้งหมดกลับไปเหมือนเดิม

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งต้องเปลี่ยนแปลงข้อมูล 2 จุดพร้อมกัน (เพิ่ม balance + บันทึกประวัติ) ถ้าไม่ใช้ Transaction แล้วเกิด error ระหว่างทาง อาจได้ข้อมูลที่เพิ่ม balance ไปแล้วแต่ไม่มีประวัติ หรือกลับกัน ทำให้ข้อมูลไม่สอดคล้องกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงทั้งหมดลงฐานข้อมูลจริงเมื่อทุกขั้นตอนสำเร็จ ส่วน ROLLBACK คือการยกเลิกและย้อนข้อมูลกลับไปสถานะก่อนเริ่ม Transaction เมื่อมีข้อผิดพลาดเกิดขึ้น

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ One-to-Many (1 Account มีได้หลาย DepositTransaction) โดยฝั่ง DepositTransaction เก็บ field อ้างอิงกลับไปหา Account แบบ Many-to-One ผ่าน foreign key account_id

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลจะไม่ถูกบันทึกเลย แม้โค้ดจะสั่ง save() ไปแล้วก่อนเกิด error ก็ตาม เพราะ Spring จะสั่ง ROLLBACK ทั้ง Transaction ทำให้ฐานข้อมูลกลับไปเหมือนสถานะก่อนเริ่มทำงาน

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อไม่มี @Transactional แต่ละคำสั่ง save() จะบันทึกลงฐานข้อมูลทันทีที่คำสั่งแยกจากกัน ไม่มีใครมองภาพรวม ดังนั้นแม้จะเกิด error ที่หลัง ข้อมูลที่บันทึกไปแล้วก่อนหน้านี้ยังคงอยู่ ไม่ถูกย้อนกลับ ทำให้ข้อมูลไม่สอดคล้องกัน